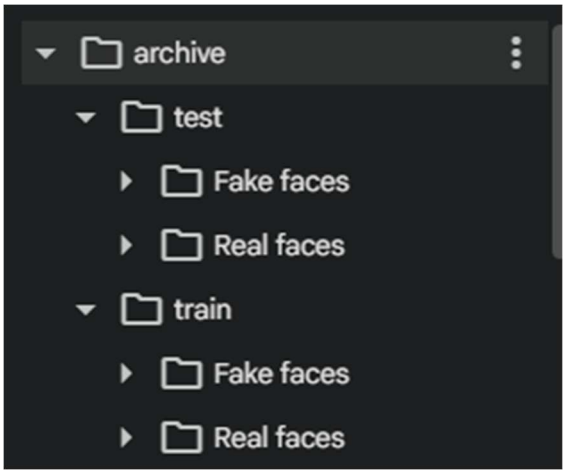
1) Upload the data from my drive and unzip it :

```
[1]
✓ 32s
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

[2]
✓ 1m
!unzip "/content/drive/MyDrive/archive.zip" -d "/content/"

  inflating: /content/archive/train/Real faces/real_9947.png
  inflating: /content/archive/train/Real faces/real_9948.png
  inflating: /content/archive/train/Real faces/real_9949.png
  inflating: /content/archive/train/Real faces/real_995.png
  inflating: /content/archive/train/Real faces/real_9950.png
  inflating: /content/archive/train/Real faces/real_9951.png
  inflating: /content/archive/train/Real faces/real_9952.png
  inflating: /content/archive/train/Real faces/real_9953.png
  inflating: /content/archive/train/Real faces/real_9954.png
  inflating: /content/archive/train/Real faces/real_9955.png
  inflating: /content/archive/train/Real faces/real_9956.png
  inflating: /content/archive/train/Real faces/real_9957.png
  inflating: /content/archive/train/Real faces/real_9958.png
```

2) The folders structure after unzip it :



3) Define and verify the paths to the dataset :

```
[3]
✓ 5s
from google.colab import drive
drive.mount('/content/drive', force_remount=True)

Mounted at /content/drive

[4]
✓ 0s
# Define paths to the dataset
train_dir = '/content/archive/train'
test_dir = '/content/archive/test'

[5]
✓ 0s
import os
print(len(os.listdir('/content/archive/train/Fake faces')))
print(len(os.listdir('/content/archive/train/Real faces')))
print(len(os.listdir('/content/archive/test/Fake faces')))
print(len(os.listdir('/content/archive/test/Real faces')))

10000
10000
10000
10000
```

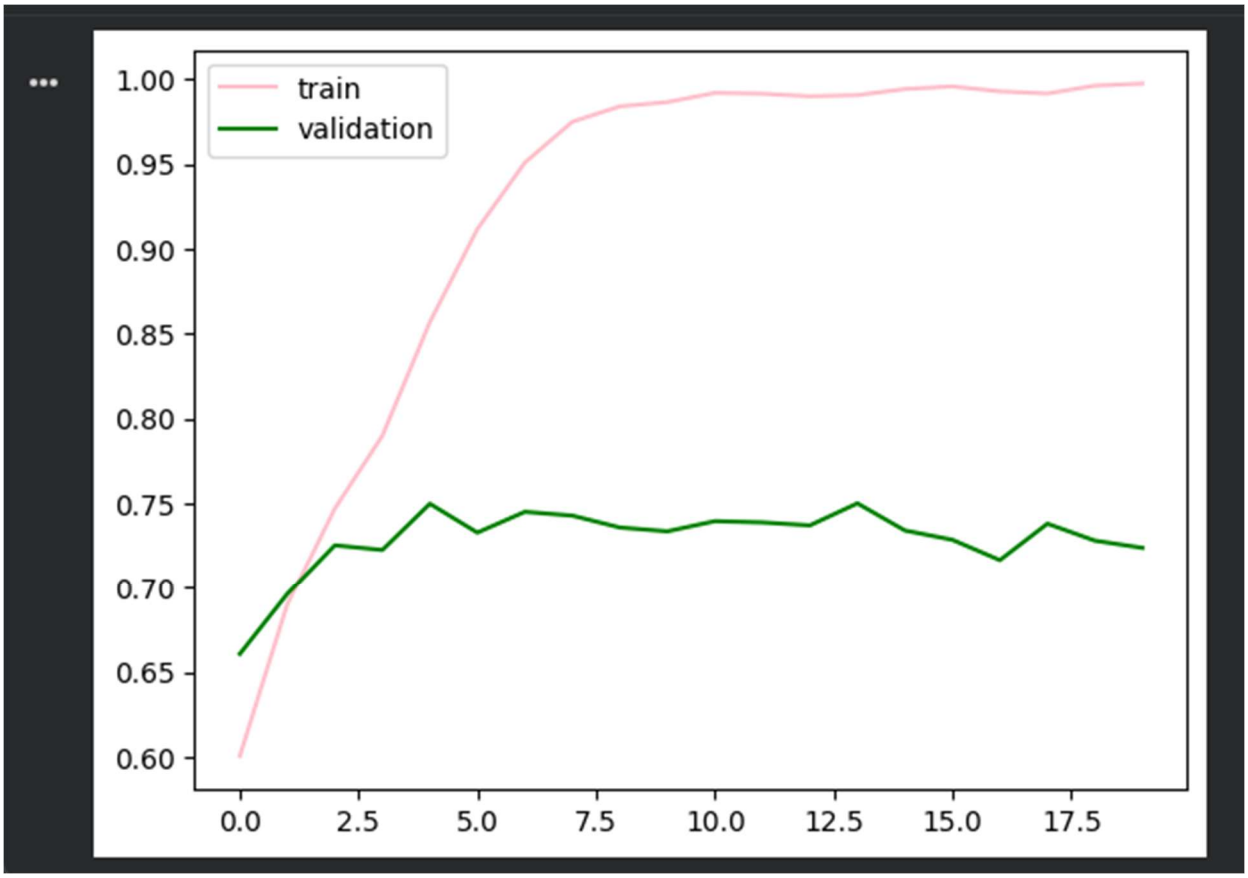
4)dataset loading :

```
...  Found 16000 images belonging to 2 classes.
     Found 4000 images belonging to 2 classes.
```

5)model training and optimization (using T4 GPU) :

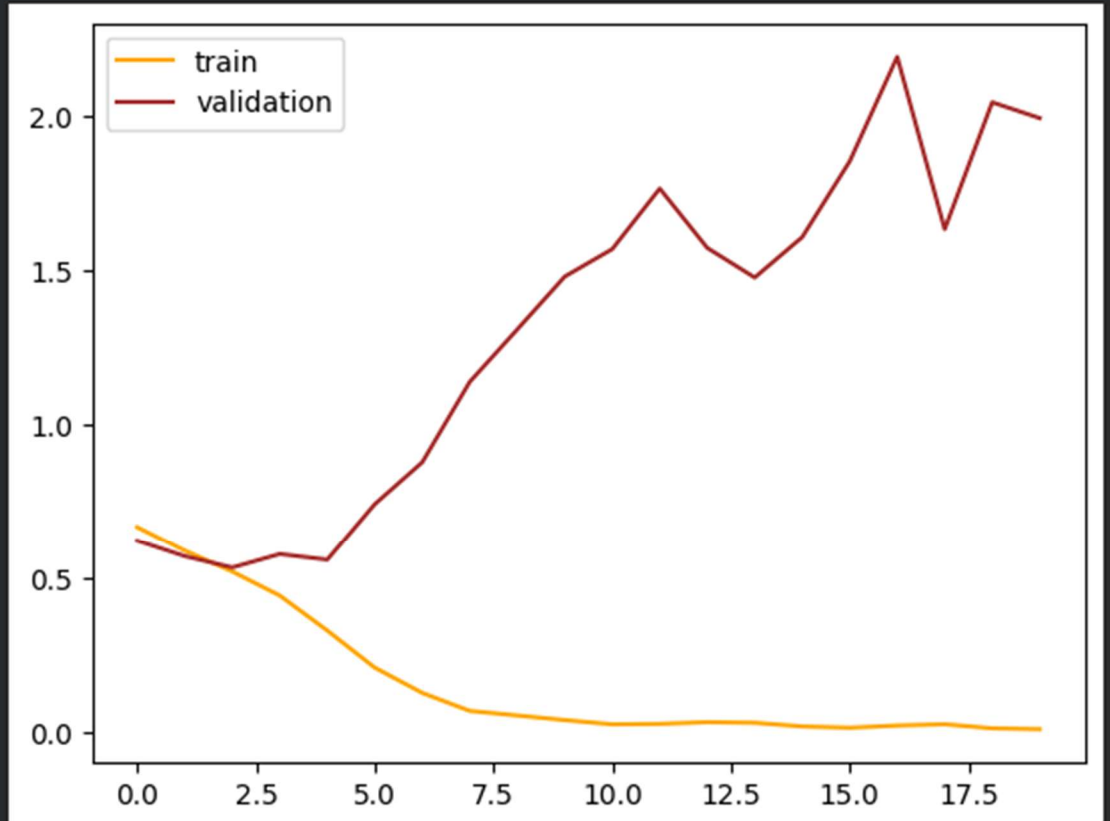
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

from keras.optimizers import Adam
model.compile(optimizer=Adam(learning_rate=0.001),loss='binary_crossentropy',metrics=['accuracy']) #binary_

history = model.fit(train_data, epochs=20, validation_data=validation_data)

Epoch 1/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 108s 197ms/step - accuracy: 0.6006 - loss: 0.6665 - val_accuracy: 0.6607 - val_
Epoch 2/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 90s 181ms/step - accuracy: 0.6902 - loss: 0.5889 - val_accuracy: 0.6963 - val_l
Epoch 3/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 90s 180ms/step - accuracy: 0.7469 - loss: 0.5206 - val_accuracy: 0.7253 - val_l
Epoch 4/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 90s 180ms/step - accuracy: 0.7898 - loss: 0.4425 - val_accuracy: 0.7225 - val_l
Epoch 5/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 88s 176ms/step - accuracy: 0.8568 - loss: 0.3290 - val_accuracy: 0.7498 - val_l
Epoch 6/20
```

6) accuracy graph :



7)loss graph :

```
plt.show()
```



8)model prediction on test dataset :

```
Found 20000 images belonging to 2 classes.
```

```
#predict the test data
predictions = model.predict(test_data)
```

```
625/625 ━━━━━━━━━━━━━━━━━━━━ 103s 165ms/step
```

9)model evaluation : confusion matrix & classification report :

```
Confusion Matrix:
[[4985 5015]
 [5005 4995]]
Classification Report:
              precision    recall  f1-score   support

           0       0.50      0.50      0.50     10000
           1       0.50      0.50      0.50     10000

    accuracy                           0.50     20000
   macro avg       0.50      0.50      0.50     20000
weighted avg       0.50      0.50      0.50     20000
```

10)model epochs monitoring and training completion :

```
Epoch 9/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 91s 181ms/step - accuracy: 0.6878 - loss: 0.6011 - val_accuracy: 0.6860 - val_l
Epoch 10/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 92s 183ms/step - accuracy: 0.6886 - loss: 0.5982 - val_accuracy: 0.6740 - val_l
Epoch 11/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 92s 184ms/step - accuracy: 0.6916 - loss: 0.5936 - val_accuracy: 0.6833 - val_l
Epoch 12/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 90s 179ms/step - accuracy: 0.7077 - loss: 0.5772 - val_accuracy: 0.6075 - val_l
Epoch 13/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 91s 182ms/step - accuracy: 0.7196 - loss: 0.5646 - val_accuracy: 0.7285 - val_l
Epoch 14/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 92s 183ms/step - accuracy: 0.7293 - loss: 0.5576 - val_accuracy: 0.6950 - val_l
Epoch 15/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 91s 182ms/step - accuracy: 0.7274 - loss: 0.5547 - val_accuracy: 0.7265 - val_l
Epoch 16/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 93s 185ms/step - accuracy: 0.7502 - loss: 0.5326 - val_accuracy: 0.6945 - val_l
Epoch 17/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 92s 183ms/step - accuracy: 0.7544 - loss: 0.5225 - val_accuracy: 0.5288 - val_l
Epoch 18/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 92s 185ms/step - accuracy: 0.7581 - loss: 0.5167 - val_accuracy: 0.7283 - val_l
Epoch 19/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 93s 186ms/step - accuracy: 0.7586 - loss: 0.5156 - val_accuracy: 0.7393 - val_l
Epoch 20/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 96s 191ms/step - accuracy: 0.7705 - loss: 0.4972 - val_accuracy: 0.7230 - val_l
```

11) loading test dataset & running final predictions :

```
[22]
✓ 0s
test_data = test_datagen.flow_from_directory(
    test_dir,
    target_size=(256,256),
    batch_size=32,
    class_mode='binary'
)

Found 20000 images belonging to 2 classes.

[23]
✓ 1m
predictions = model.predict(test_data)

625/625 ━━━━━━━━━━━━━━━━━━━━ 84s 134ms/step
```

12) classification report after overfitting mitigation :

```
Confusion Matrix:
[[3481 6519]
 [3549 6451]]
Classification Report after applying techniques to handle overfitting:
              precision    recall  f1-score   support

           0       0.50      0.35      0.41     10000
           1       0.50      0.65      0.56     10000

    accuracy                           0.50     20000
   macro avg       0.50      0.50      0.49     20000
weighted avg       0.50      0.50      0.49     20000
```

13)downloading pre-trained ResNet50 weights & fine-tuning :

```
••• Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights
94765736/94765736 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

14)running inference and test predictions with ResNet50 :

```
Epoch 1/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 118s 207ms/step - accuracy: 0.5440 - loss: 0.8641 - val_accuracy: 0.5475 - val_
Epoch 2/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 97s 194ms/step - accuracy: 0.5755 - loss: 0.6982 - val_accuracy: 0.5635 - val_l
Epoch 3/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 97s 194ms/step - accuracy: 0.5853 - loss: 0.6781 - val_accuracy: 0.6330 - val_l
Epoch 4/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 97s 194ms/step - accuracy: 0.5929 - loss: 0.6667 - val_accuracy: 0.5675 - val_l
Epoch 5/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 98s 195ms/step - accuracy: 0.5987 - loss: 0.6628 - val_accuracy: 0.6217 - val_l
Epoch 6/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 99s 198ms/step - accuracy: 0.6074 - loss: 0.6556 - val_accuracy: 0.5543 - val_l
Epoch 7/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 96s 191ms/step - accuracy: 0.6026 - loss: 0.6568 - val_accuracy: 0.5947 - val_l
Epoch 8/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 95s 190ms/step - accuracy: 0.6002 - loss: 0.6605 - val_accuracy: 0.5155 - val_l
Epoch 9/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 94s 188ms/step - accuracy: 0.5876 - loss: 0.6673 - val_accuracy: 0.5717 - val_l
Epoch 10/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 95s 189ms/step - accuracy: 0.5859 - loss: 0.6680 - val_accuracy: 0.5555 - val_l
Epoch 11/20
500/500 ━━━━━━━━━━━━━━━━━━━━ 94s 188ms/step - accuracy: 0.5882 - loss: 0.6653 - val_accuracy: 0.5985 - val_l
```

15)final performance evaluation of ResNet50 model :

```
predictions = resnet_model.predict(test_data)

625/625 ━━━━━━━━━━━━━━━━━━━━ 98s 149ms/step
```

16)ResNet50 model classification report & final evaluation metrics :

```
Confusion Matrix:
[[6957 3043]
 [6910 3090]]
Classification Report after applying techniques to handle overfitting:
              precision    recall  f1-score   support

           0       0.50      0.70      0.58     10000
           1       0.50      0.31      0.38     10000

    accuracy                           0.50     20000
   macro avg       0.50      0.50      0.48     20000
weighted avg       0.50      0.50      0.48     20000
```

17) final workspace and local file management :